

Claude Code: Principles

Dave Figueroa

Working principles from hands-on Claude Code configuration across multiple projects, developed through my CodeGarage — a meta-project for auditing, tuning, and incubating Claude Code infrastructure.

Configuration is the work of giving Claude what it needs to do good work: orientation, judgment-bearing context, and the right tool at the right layer. Claude has strong defaults; configuration earns its keep by directing where defaults aren't enough, not by cataloguing what to avoid. Every line in front of Claude shapes what it attends to; lines that don't shape attention dilute the ones that do.

Throughout, role names like *coordinator*, *builder*, and *evaluator* refer to custom subagents from my pipelines: conventions of mine, not Claude Code primitives.

I. Context Economics

1. Dilution Is the Failure Mode

Everything in context competes for attention. The failure mode isn't just wasted tokens — it's dilution. Larger context windows don't relieve dilution; they amplify it, because the visible cost of adding a rule keeps falling while the attention cost doesn't. A long list of "don't do X" rules becomes background noise by the time the specific edge case arrives. The model stops distinguishing between critical guardrails and cautionary notes. Every turn an agent spends processing unnecessary context is API spend that produced no judgment value. Context efficiency is simultaneously a quality, reliability, and cost concern.

2. The Single Line Test

Every line in your configuration should pass one test: **"Would removing this line cause Claude to make a mistake?"** If not, cut it. The test's strictness varies by layer. Root CLAUDE.md is strictest: it loads for every session, every agent. An agent prompt that loads once for a single judgment-heavy session has a different cost calculus; comprehensiveness there is often correct.

3. Bloat Costs More at the Top

CLAUDE.md is the most expensive real estate; agent prompts are the cheapest. Be strict where context loads always, comprehensive where it loads once. A lean CLAUDE.md frees budget for depth in agent prompts, and that's where it matters most. Agent prompts for judgment-heavy, single-purpose agents are the highest-return investment in your entire configuration.

4. Stale Configuration Is Worse than No Configuration

Claude reads everything in CLAUDE.md as ground truth. When status-oriented facts drift, Claude makes decisions based on a world that no longer exists. No status leaves Claude uncertain; stale status makes Claude confidently wrong. The fix is separating what changes from what doesn't. Architecture truths (pipeline shape, module ownership) go in CLAUDE.md. Status (phase, what's built, what's pending) needs a different home, ideally one mechanically updated by the pipeline.

II. Layer Discipline

5. Find the Right Layer

When a problem surfaces, find the most structural layer that solves it and solve it there. The layers:

1. **Structure:** hooks, tool restrictions, file permissions
2. **Process design:** separation of concerns between agents
3. **Agent instructions:** focused on judgment and sequencing
4. **Path-scoped rules:** behavioral rules that load only when relevant
5. **Ambient rules:** behavioral expectations that load every session
6. **CLAUDE.md:** project orientation (most dilution-prone)

If a builder keeps editing tests, that needs a hook (layer 1), not an instruction (layer 3). Responding to recurring problems with more instructions instead of structural enforcement is the most common mistake.

When the problem is *orchestration itself* (what runs next, in what order, with what retries), the most structural option is to hold the control flow in a deterministic script rather than an agent's turn-by-turn judgment. Orchestration-as-code outranks separation-by-convention (2) and coordinator instructions (3): a script can't be reasoned past, and it can't silently decide to do the work itself. See III for where it lands on the judgment/mechanism axis.

6. Just-in-Time Knowledge

Instructions are most effective when they arrive at the moment they're contextually relevant, not preloaded as permanent weight. Path-scoped rules load when Claude touches matching files. Skills load when invoked. Agent prompts deliver context at spawn time. A scoped rules file for one domain doesn't eat context during work in another.

7. Hooks Can't Be Reasoned Past

A hook that blocks an action cannot be reasoned past; an instruction that says "don't do X" can. When a behavioral rule keeps getting violated, promoting it to a structural hook is the next step. Coaching feedback (#11) in the hook output makes this more effective than silent denial. The agent gets actionable feedback without trial and error.

8. CLAUDE.md Answers One Question

CLAUDE.md answers: "**What is this project and where are things?**" Project identity, architecture pointers, fragile areas. Behavioral rules, domain knowledge, and process instructions all have better homes: places where they load conditionally or stay templatable across projects.

III. Judgment vs. Mechanism

The axis here has two endpoints: a script (deterministic, no judgment) and a skill (an LLM in the loop because the right action depends on reading the situation). A third point sits between them. A script whose *steps* are judgment calls is a deterministic orchestration of non-deterministic agents: the sequencing (what runs, in what order, how many retries) is code, so it can't be reasoned past or silently adapted, while the work inside each step is still an agent exercising judgment. Put the determinism in the sequencing and the judgment in the steps.

9. The Token Cost of a Skill Is the Cost of Discernment

A script is cheaper, faster, fully deterministic. The reason to pay the token cost of an LLM in the loop is that the right action depends on reading the situation. A commit skill that examines the diff and writes an appropriate message does something `git commit -m "update"` never could. A test runner that just executes `pytest` doesn't need to be a skill. Match the mechanism to the judgment required.

10. Automate the Mechanical, Reserve Turns for Judgment

When an agent spends half its turns running boilerplate that could be a script or skill, that's three costs at once: split attention (clarity loss), context filled with mechanical output (quality loss), and API spend with no judgment value (cost). The best skills *use* scripts for their mechanical steps and reserve LLM attention for the judgment parts. Clarity, quality, and cost all point the same direction.

11. Coaching Beats Silent Blocking

A hook that blocks an action and tells the agent why is significantly more effective than silent denial. A silent block on a raw `git commit` sends the agent looking for a different path: staging manually, trying alternate syntax, eventually finding some way to commit that still bypasses the skill. "Blocked: use the `/commit` skill, which examines the diff and follows project conventions" sends the agent directly to the right tool. The block is still deterministic; the coaching just makes it useful.

12. Invest Where Judgment Compounds

Comprehensiveness isn't dilution when every line earns judgment. The cost calculus differs by where the prompt sits. Three categories matter:

- **Judgment multipliers** (planners, evaluators): comprehensive prompts raise the quality ceiling for every downstream agent and product produced. Invest deeply.
- **Judgment endpoints** (doc-curator): no quality gate downstream. The bar must be internalized.
- **Execution agents** (builders, verifiers): can stay lean if the judgment agents upstream are strong.

IV. Autonomy & Safety

13. Guard the Cliffs, Not the Road

If failure is detectable and recoverable, permissiveness is preferred. If failure is irreversible, deny by default.

Permissiveness buys two things: visibility and adaptability. Mistakes are evidence. A builder that modifies files it shouldn't shows you the pattern to fix structurally, which a tight constraint would have hidden. And capable models bring judgment to edge cases the rules didn't anticipate, find better paths, recover from surprises, but only when they have room to do so. Constraints have costs that compound as models get more capable; the more able the agent, the more you forfeit by hemming it in. The cliffs — `git push --force`, `rm -rf` on untracked work — get explicit denies because no downstream agent can undo them. Everything else, you let run and watch.

14. Adaptiveness Without Observability Is a Liability

Claude's adaptiveness is also its most dangerous quality in a pipeline. When something fails, Claude doesn't stop and report. It adapts. A coordinator invoked the wrong way silently does everything itself, bypassing the evaluator and every quality gate. A broken skill silently falls back to raw bash, bypassing every check the skill enforced. Silent adaptation defeats detection, turning recoverable failures into undetectable ones. Pipeline design must build detection for the adaptation, not just the original failure.

15. Surface the Adaptation

Adaptation isn't the problem; silent adaptation is. An agent that reports "I expected X, found Y, adjusted by doing Z" or "I tried X, it didn't work because Y" is doing the work as designed; reporting deviations is part of the work, not overhead. An agent that quietly produces output that *looks like the plan worked* defeats every downstream check. As agents get more capable and pipelines more autonomous, what they report back becomes the load-bearing piece keeping the human in the loop.

16. Knowing When to Quit

Claude is trained to deliver, and can almost always find a workaround, which is exactly the problem. The discipline isn't recognizing when the agent *can't* push through; it's establishing when it *shouldn't*. Asked to build X with Y, if Y is broken, a tragically helpful agent silently scopes to "X without Y" and delivers a "success" the user can't course-correct because they can't see it. When a workaround would change the deliverable's identity, silently reduce scope, or paper over a problem the user needs to see, the right move is to stop and report. Surfacing real problems is part of the work, not a fallback from it. An honest stop preserves the human's ability to decide; a tragic completion forecloses it.

17. The Human Is for Decisions, Not Mechanical Approval

The high-value human work is planning, scoping, and deciding when work is ready. With autonomous execution available, the human's role is increasingly checkpoint-shaped: verifying outputs, approving promotion, answering scoping questions, not approving routine tool calls in real time. Structural enforcement (deny rules, scoped tools, hooks) handles the safety layer so attention stays on decisions only a human can make. `bypassPermissions` isn't the answer for getting out of the way; tightening the structure is.

V. Separation of Concerns

18. The Strongest Rule Is a Withheld Tool

An agent that doesn't have the Write tool can't edit code, regardless of what its prompt says. An evaluator with read-only tools can't be talked into a quick fix mid-review. A coordinator without Write/Edit can't decide to "just handle it itself" when delegation feels slow. Behavioral separation enforced by instruction is a suggestion; behavioral separation enforced by tool configuration is a constraint. It's #7 at the agent level: instructions are suggestions, tool restrictions are constraints.

19. Specialize When Incentives Diverge

An agent doing both build and evaluation has incentive to grade its own work generously, especially when the alternative is starting over. The builder optimizes for shipping; the evaluator optimizes for catching. Combine them and you get the average of the two pressures. Separation isolates the pressures so each agent is judged on what it's actually built to do. The principle generalizes: split agents wherever combining roles would produce a conflict the pipeline can't audit, because the compromise happens inside the agent's reasoning, not in any artifact a downstream check can inspect.

20. Each Handoff Is a Filter

A handoff compresses what the previous agent saw into what they returned: the next agent gets the output, not the reasoning. That's often the point. A researcher agent mines logs and hands back the finding; a doc-curator trims library docs to what matters. The compression is the value. But the same mechanism can drop signal. A builder hands off to an evaluator without conveying why an approach was chosen, and the evaluator can't tell intentional choice from oversight.

Place each boundary where its filter drops noise to amplify signal. The failure runs in both directions: too few agents and structural enforcement falls apart; too many and signal degrades, because no filter is perfect and there are only so many natural points where filtering helps. Each handoff also pays costs in spawn overhead and re-orientation. The right granularity is the smallest number of agents that gives you the capability separations you actually need. Coordinator + builder + evaluator covers most cases. Add a planner when planning is its own judgment-heavy step. Add a verifier when "did the build pass?" is a separable question from "is the work good?"

VI. Pipeline Contracts

21. Define Done, or Claude Will

Claude fills gaps well, so a missing definition of done never stalls the work. The agent supplies its own. It decides which tests count, what the PR includes, what "passing" means, and ships against that. Plausible, confident, and not yours. A fuller description won't fix it: prose criteria get quietly narrowed to fit whatever got built, and the narrowing reads as success. A check can't be narrowed after the fact. It clears the bar you set or it doesn't. Encode done as a test, a check, a CI gate the agent has to turn green; make it prove done, not claim done.

Every other layer raises the probability of good work; a definition of done you can check confirms it.

22. Acceptance Criteria Are an Autonomy Contract

When a pipeline runs autonomously, criteria become load-bearing. Vague criteria cause every downstream agent to guess, and guessing compounds. Four checks for every criterion, testable first:

- **Testable:** a machine can verify it automatically, as a test assertion
- **Observable:** a human can verify it after the fact, by running a command or reading output
- **Bounded:** scope is clear
- **Unambiguous:** two reasonable agents wouldn't disagree

23. Concrete Boundaries Are Greppable Boundaries

Every boundary rule must be checkable by grepping imports or reading a diff. "No sqlite3 imports outside db.py and test files" passes: an evaluator can grep for it. "Use modular architecture" fails: an evaluator can't verify it against a diff. If you can't describe a grep command or point to a specific import to check, the rule isn't concrete enough. Rules files are the evaluator's enforcement spec; their concreteness bounds its effectiveness.

24. User Input Is for Prescribed Checkpoints

When an agent stops to ask the user something mid-execution, two upstream failure modes hide behind the same symptom: a vague criterion that should have been tightened in planning, or a decision gate that should have been scheduled as a checkpoint. Neither is a feature to build by adding "ask user" capability; both are signals that structure is missing upstream. Letting agents ask freely erodes the autonomy contract: every interruption is a context switch for the human, and once asking is permitted, planning quality degrades because gaps don't have to be resolved before launch. Surfacing what changed (#15) is fine: that's reporting, not blocking. Quitting cleanly on a problem (#16) is reporting too. Asking is for checkpoints.

The fix is never to suppress questions; it is to move them earlier, into planning, where the human already spends the bulk of their attention. This holds at every agentic depth: a direct session and a deep pipeline both run cleaner when the gaps are closed before the build phase starts. A question answered in planning costs one exchange; the same question surfacing mid-build costs a stop, a redo, and whatever was already built on the wrong assumption. Plan in the fullest detail the task allows. The thoroughness of the plan, not the permissiveness of asking, is what keeps the build phase from stalling, and mid-execution questions are the symptom that planning left something unresolved.

VII. Memory

25. Memory Is Claude's Domain, but Your Responsibility

Memory writing is autonomous: Claude writes during directed work, shaped by what felt important in the moment, potentially without you noticing. You don't always choose what gets saved; the default flow doesn't route through you. The leverage is direction, not suppression. Trigger cleanup yourself; Claude won't initiate it during directed work, so memories accumulate and degrade until something prompts a review. Whether to run memory at all is an up-front, workflow-dependent call: it lives in interactive planning and discussion, so if the upkeep outweighs the gain for how you work, disabling it is legitimate.

26. The Memory Index Is for Routing

The memory index loads every session; the underlying memory files don't. Each entry's description is what tells Claude whether to open the file, which makes descriptions the routing mechanism. Write them as activation context ("when should I open this?"), not as content summaries. A summary tells Claude what's in the file; a trigger tells Claude when it matters. The first makes the index a table of contents; the second makes it a triage layer.

27. Memory Is the Configuration That Rots

Every other layer changes only when you change it. Memory writes itself, accruing as Claude works, with nothing to maintain what accumulates. It is hidden context that goes stale untouched: nothing corrects a claim that stops being true. A stale memory may override the instruction you deliberately wrote; you cannot count on the authored rule winning. And because it may not fire every session, the interference could be intermittent: the hardest kind to catch, because it will not reproduce on demand. The staleness you cannot see is the staleness that gets you.

28. Mark What Will Go Stale

Every claim in a memory is durable or temporal. **Durable facts** are true about the nature of something: "this project is a portfolio site," "the user prefers rebase over merge." Safe to trust without verification. **Temporal claims** are true at a point in time: status updates, file paths, "next session" plans. Mark them explicitly so they can be discarded. Autonomous memory maintenance requires an explicit trigger; left to itself, memory accumulates and degrades indefinitely.

VIII. Configuration Lifecycle

29. Write Rules After Mistakes, Not Before

A rule written after a mistake is a record of an observed failure; a rule written before one is a guess, and Claude's defaults are strong enough that most guessed failures never arrive. Speculative rules are how a configuration bloats: each looks prudent in isolation, none can be checked against anything that actually happened, and together they dilute the rules that earn their keep. A rule born from a real mistake has what a guessed one lacks: evidence it's needed, a concrete behavior to watch for recurrence, and a visible condition for retiring it when the model stops making the mistake. So wait for the failure, then fix it at the most structural layer that solves it. This is what guarding only the cliffs (#13) buys: with the irreversible denied up front, every mistake that remains is recoverable, and a recoverable mistake is cheap evidence.

30. Cost Is Visible, Use Isn't

You can see what a component costs to load, but not how often it earns that cost. The asymmetry breeds bloat: the cost is paid every session whether the component is used or not, and nothing surfaces the difference. The Single Line Test judges each line by inspection, but inspection can't see usage. Per-component instrumentation can, measuring cost per skill, subagent, rule file, and MCP server alongside how often each is actually used. It replaces "I think this dilutes" with what a component costs and whether that cost is paying off. And it catches what inspection misses, because the heaviest waste is rarely the lines under scrutiny: a forgotten MCP server, loaded every session and rarely used, outweighs the CLAUDE.md prose everyone edits. Measure before you assert, and re-measure after you cut.

31. Re-tune as the Model Improves

Every guardrail is written against a specific failure of a specific model, and the model underneath keeps changing: each generation absorbs judgment its predecessor needed scaffolding for. A constraint that earned its keep against one model's mistakes can be dead weight against its successor's, and dead weight is dilution on a slower clock than stale status. Where #4 watches status facts decay, this watches guardrails decay: the mistake that justified a rule stops happening, the rule stays, and it is now calibrated to a model that no longer exists. So re-audit the guardrails on every model upgrade, not just the facts: which mistakes does it no longer make, and which constraints now cost more than they prevent? A configuration that only ever grows is one nobody has re-audited against the model they actually have.

Feedback *and* *corrections* *welcome* *via*
[issues](https://github.com/daedahl/claude-code-principles/issues).

v1.1 · © Dave Figueroa · [CC BY 4.0](LICENSE)